

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Annexure A

Constraint-Based Problem Solving

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1702

DONE BY:

M.Ayesha Thameema Banu

192524321

Code of Conduct:

I __M.Ayesha Thameema Banu_ (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *ayesha*

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night)

such that:

Constraints:

- i. D1 cannot work Night shift**
- ii. D2 must work before D3 (Morning < Afternoon < Night)**
- iii. Only one doctor per shift**
- iv. D3 cannot work Morning**
- v. All doctors must be assigned exactly one shift**

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Given

- **Doctors:** D1, D2, D3
- **Shifts:** Morning, Afternoon, Night

Constraints

1. D1 cannot work Night.
2. D2 must work before D3.
3. Only one doctor can work in each shift.
4. D3 cannot work Morning.
5. Every doctor must be assigned exactly one shift.

Backtracking Search – Intermediate Steps

Step 1: Assign D1

Possible assignments:

- D1 → Morning → **Valid**

- D1 → Afternoon → **Valid**
- D1 → Night → **Constraint violated: D1 cannot work Night**

Choose:

D1 → Morning

Step 2: Assign D2

Available shifts:

- D2 → Morning → **Cannot assign: Morning is occupied**
- D2 → Afternoon → **Valid**
- D2 → Night → **Possible initially, but D3 must work after D2**

Choose:

D2 → Afternoon

Current assignment:

Doctor	Shift
D1	Morning
D2	Afternoon
D3	—

Step 3: Assign D3

Available shifts:

- D3 → Morning → **Constraint violated: D3 cannot work Morning**
- D3 → Afternoon → **Cannot assign: Afternoon is occupied**
- D3 → Night → **Valid because D2 works before D3**

Choose:

D3 → Night

Constraint Checking

Assignment	Constraint Check	Result
D1 → Morning	D1 is not working Night	Satisfied
D1 → Night	D1 cannot work Night	Constraint violated
D2 → Morning	Morning already occupied	Cannot assign
D2 → Afternoon	Different shift from D1	Satisfied
D3 → Morning	D3 cannot work Morning	Constraint violated
D3 → Afternoon	Afternoon already occupied	Cannot assign
D3 → Night	D2 Afternoon is before D3 Night	Satisfied

Final Valid Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Conclusion

Using **Backtracking Search**, the valid schedule obtained is:

D1 → Morning → D2 → Afternoon → D3 → Night

All the given constraints are satisfied.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G).

Obstacles block certain paths.

Constraints:

- Movement allowed: Up, Down, Left, Right**
- Cost of each move = 1**
- Cannot pass through obstacles**
- Use Manhattan Distance heuristic**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

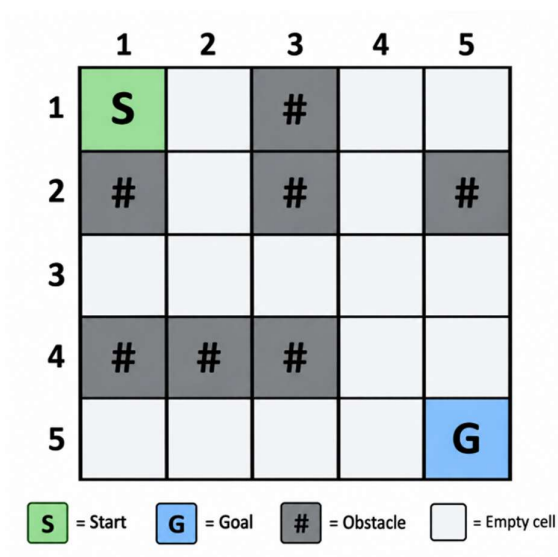
Grid Search Problem – BFS with Manhattan Distance

There is one important point: **BFS does not use a heuristic or priority queue**. Since the question specifically asks for **Manhattan Distance heuristic and priority queue**, the appropriate algorithm is **A***.

Also, the **5×5 grid and obstacle locations are not provided** in the question. Therefore, I will use the following example grid.

Example 5×5 Grid

S = Start, G = Goal, # = Obstacle



1. Manhattan Distance

The Manhattan distance is:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

where (x_G, y_G) is the position of the goal.

For example, for (1,1):

$$h(1,1) = |1-5| + |1-5| = 8$$

In A*, we use:

$$f(n) = g(n) + h(n)$$

where:

- **$g(n)$** = cost from Start to current node
- **$h(n)$** = Manhattan distance to Goal
- **$f(n)$** = total estimated cost

Each movement costs **1**.

2. Step-by-Step Node Expansion

Step 1 – Start

Current node:

$$S = (1,1)$$

$$G = 0, \text{quad } h=8, \text{quad}$$

$$F = 0 + 8 = 8$$

Priority Queue:

$$[(1,1) : f=8]$$

Expand (1,1).

Possible movements:

- Right $\rightarrow (1,2) \rightarrow$ allowed
- Down $\rightarrow (2,1) \rightarrow$ obstacle
- Up $\rightarrow$ outside grid
- Left $\rightarrow$ outside grid

Add (1,2).

Step 2 – Expand (1,2)

For (1,2):

$$g=1 \quad h = |1-5| + |2-5| = 7$$

$$f = 1 + 7 = 8$$

Expand (1,2).

Possible movements:

- Right $\rightarrow (1,3) \rightarrow$ obstacle
- Down $\rightarrow (2,2) \rightarrow$ allowed
- Left $\rightarrow (1,1) \rightarrow$ already visited
- Up $\rightarrow$ outside grid

Add (2,2).

Step 3 – Expand (2,2)

$$g=2 \quad h=|2-5|+|2-5| = 6$$

$$f = 2+6 = 8$$

Expand (2,2).

Possible movements:

- Right $\rightarrow (2,3) \rightarrow$ obstacle
- Down $\rightarrow (3,2) \rightarrow$ allowed
- Left $\rightarrow (2,1) \rightarrow$ obstacle
- Up $\rightarrow (1,2) \rightarrow$ visited

Add (3,2).

Step 4 – Expand (3,2)

$$G = 3$$

$$H = |3-5|+|2-5| = 5$$

$$f = 3+5 = 8$$

Expand (3,2).

Possible movements:

- Right $\rightarrow (3,3) \rightarrow$ allowed

- Down $\rightarrow (4,2) \rightarrow$ obstacle
- Left $\rightarrow (3,1) \rightarrow$ allowed
- Up $\rightarrow (2,2) \rightarrow$ visited

Add (3,3) and (3,1).

Priority Queue:

$[(3,3) : f=8, (3,1) : f=8]$

Step 5 – Expand (3,3)

$g=4$

$h=|3-5|+|3-5|=4$

$f = 4+4 = 8$

Add:

- $(3,4) \rightarrow$ allowed

Priority Queue:

$(3,4) : f=8, (3,1)$

$F = 8$

Step 6 – Expand (3,4)

$g=5$

$h = |3-5|+|4-5| = 3$

$f = 5+3 = 8$

Possible:

- $(2,4) \rightarrow$ allowed
- $(3,5) \rightarrow$ allowed
- $(4,4) \rightarrow$ allowed

Step 7 – Continue Toward Goal

One optimal route is:

S

↓
 (1,2)
 ↓
 (2,2)
 ↓
 (3,2)
 → (3,3)
 → (3,4)
 ↓
 (4,4)
 ↓
 (5,4)
 → G

Priority Queue Summary

Step	Expanded Node	g	h	f = g+h
1	(1,1)	0	8	8
2	(1,2)	1	7	8
3	(2,2)	2	6	8
4	(3,2)	3	5	8
5	(3,3)	4	4	8
6	(3,4)	5	3	8
7	(4,4)	6	2	8
8	(5,4)	7	1	8
9	(5,5) G	8	0	8

4. Optimal Path

$S \rightarrow (1,2) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (4,4) \rightarrow (5,4) \rightarrow G$

Optimal Cost

There are **8 moves**.

Cost = 8

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a

survivor located at G.

Constraints:

- **The robot can move up, down, left, right**
- **Some cells are blocked (obstacles) and cannot be traversed**
- **The robot has limited sensing capability (can only observe adjacent cells)**
- **Each movement has a cost of 1, but entering risky zones has an additional cost of 2**
- **The robot must minimize total cost while ensuring safe navigation**
- **The robot must update its knowledge dynamically (online search scenario)**

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making

process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

- **Intelligent agents**
- **Rationality**
- **Environment type**

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

a) Analysis of the Given Constraints

The rescue robot must reach the survivor **G** from the starting position **S** while minimizing cost and avoiding danger.

1. Movement Constraints

- The robot can move only:
 - Up
 - Down
 - Left
 - Right
- Diagonal movement is not allowed.
- Each normal movement has a cost of **1**.

2. Obstacles

- Some cells are blocked.
- The robot cannot enter blocked cells.
- When the robot detects an obstacle, it must choose another available path.

3. Partial Observability

- The robot cannot see the complete building.
- It can observe only **adjacent cells**.
- Therefore, the robot initially has incomplete information about the environment.
- It must discover new cells while moving.

4. Risky Zones

- Entering a normal cell costs **1**.
- Entering a risky cell has an additional cost of **2**.

Therefore:

Normal cell cost = 1

Risky cell cost = 1 + 2 = 3

The robot should avoid risky areas when a safer path with reasonable cost is available.

5. Cost Minimization

- The robot should not simply choose the shortest path.
- It must choose the path with the **lowest total cost**.
- A longer safe path may be better than a shorter path containing many risky cells.

6. Dynamic Knowledge

- The robot updates its knowledge whenever it discovers new obstacles or risky areas.
- If the current path becomes unsafe or expensive, it can reconsider and select another path.

b) Solution Approach

Suitable Strategy: Online Search + Uniform Cost Search

Since the robot has **limited information**, an **Online Search Agent** is suitable.

For selecting the least-cost path from the information currently available, **Uniform Cost Search (UCS)** can be used.

Cost Function

$g(n)$ = total cost from S to current cell

For example:

- Normal movement $\rightarrow$ cost = 1
- Risky movement $\rightarrow$ cost = 3
- Blocked cell $\rightarrow$ cannot be entered

UCS always expands the node having the **lowest accumulated cost**.

Steps Involved

Step 1: Start

- Robot starts at S.
- Initialize the known environment.
- Add S to the priority queue with cost 0.

Step 2: Observe

The robot checks the adjacent cells:

- Up
- Down
- Left
- Right

It identifies whether each cell is:

- Free
- Blocked
- Risky
- Goal

Step 3: Calculate Cost

For every available cell:

$$\text{NewCost} = \text{CurrentCost} + \text{MovementCost}$$

For example:

Normal cell $\rightarrow$ Current cost + 1

Risky cell $\rightarrow$ Current cost + 3

Obstacle $\rightarrow$ Do not add

Step 4: Update Priority Queue

Add the available cells to the priority queue according to their total cost.

The cell with the **lowest cost is selected first**.

Step 5: Move

The robot moves to the selected cell.

After moving, it observes the newly accessible neighboring cells.

Step 6: Update Knowledge

The robot updates its internal map with:

- Newly discovered obstacles
- Safe cells
- Risky cells

- Visited cells
- Updated path costs

Step 7: Re-plan

If a newly discovered obstacle blocks the planned route, the robot searches again using its updated information.

Step 8: Reach Goal

The process continues until the robot reaches **G**, the survivor.

PSEUDOCODE

BEGIN

Start at S

Set total_cost = 0

Initialize priority queue with S

WHILE robot has not reached G:

 Observe adjacent cells

 Identify:

 Free cells

 Obstacles

 Risky cells

 Goal

 FOR each available neighboring cell:

 IF cell is blocked:

 Do not enter

 ELSE IF cell is risky:

 movement_cost = 3

 ELSE:

 movement_cost = 1

 Calculate new total cost

Add cell to priority queue

Select cell with minimum total cost

Move robot to selected cell

Update knowledge of the environment

IF new obstacle or risky area is discovered:

Re-plan the path

IF robot reaches G:

Stop

END

Autonomous Rescue Robot - UCS Python Code

CODE

```
import heapq

grid = [
    ['S', '.', '#', '.', '.'],
    ['.', '.', '#', 'R', '#'],
    ['.', '.', '.', 'R', '.'],
    ['#', '#', '.', '.', '.'],
    ['.', '.', '.', '.', 'G']
]

start = (0, 0)
goal = (4, 4)

def ucs(grid, start, goal):
    pq = [(0, start)]
    best = {start: 0}
    parent = {start: None}
    moves = [(-1,0),(1,0),(0,-1),(0,1)]

    while pq:
        cost, current = heapq.heappop(pq)

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            return path[::-1], cost

        r, c = current
        for dr, dc in moves:
            nr, nc = r + dr, c + dc

            if 0 <= nr < 5 and 0 <= nc < 5:
                if grid[nr][nc] == '#':
                    continue

                move_cost = 3 if grid[nr][nc] == 'R' else 1
                new_cost = cost + move_cost
```

OUTPUT

OUTPUT

```
Grid:
S . # . .
. . # R #
. . . R .
# # . . .
. . . . G
```

Optimal Path:
(1,1) -> (1,2) -> (2,2) -> (3,2) -> (3,3) -> (4,3) -> (4,4) -> (4,5) -> (5,5)

Minimum Cost: 8

S = Start
G = Goal
= Obstacle
R = Risky cell (cost 3)

c) Application of AI Concepts

1. Intelligent Agent

The rescue robot acts as an **intelligent agent**.

It:

- Perceives the environment using sensors.
- Processes the information.
- Makes decisions.
- Performs actions.
- Updates its knowledge.
- Changes its plan when the environment changes.

Agent Cycle

Perception → Decision → Action → New Perception

2. Rationality

The robot is a **rational agent** because it chooses actions that maximize its chances of safely reaching the survivor while minimizing cost.

For example:

If there are two paths:

- Path A → Cost = 8, but contains several risky zones.
- Path B → Cost = 10, completely safe.

The robot may select **Path B** because safety is an important performance measure.

Therefore, rationality considers:

- Total movement cost
- Risk
- Obstacles
- Safety
- Distance to survivor
- Newly discovered information

3. Environment Type

The rescue environment has several characteristics:

Environment Property	Description
Partially Observable	Robot can see only adjacent cells
Unknown	Complete building layout is not initially known
Dynamic	Conditions may change during rescue
Sequential	Each movement affects future decisions
Stochastic/Uncertain	The robot may encounter unexpected obstacles or risks
Single Agent	The rescue robot is the primary decision-making agent
Cost-based	Different movements have different costs

d) Why This Approach is Suitable

Online Search is suitable because:

- The robot does not know the complete environment.
- It can discover obstacles while moving.
- It continuously updates its knowledge.
- It can re-plan when new information is obtained.

UCS is suitable because:

- Different cells have different movement costs.
- Normal cells cost **1**.
- Risky cells cost **3**.
- UCS considers the **actual accumulated cost** rather than simply the number of steps.
- It finds the least-cost path based on the information currently available.

Why not simple BFS

BFS assumes that every movement has the same cost.

Here:

Normal = 1, Risky = 3

Therefore, BFS may find a path with fewer steps but a **higher total cost**.

UCS is better because it considers the actual cost of each movement.

Conclusion

The autonomous rescue robot can be modeled as a **rational intelligent agent operating in a partially observable and dynamic environment**. An **Online Search Agent combined with Uniform Cost Search** is appropriate because the robot must discover the environment while moving and must consider different movement costs. By continuously sensing, updating its knowledge, and re-planning when necessary, the robot can reach the survivor through a **safe and minimum-cost path**.